

Laporan Praktikum Kontrol Cerdas

Nama : Fryscha Febryanti Puspitasari

NIM : 224308008

Kelas : TKA 6A

Akun Github (Tautan) : <https://github.com/Fryscha>

Student Lab Assistant : Yulia Brilianty

1. Judul Percobaan

Canny Edge Detection & Lane Detection With Instance Segmentation

2. Tujuan Percobaan

Percobaan ini dilakukan dengan tujuan sebagai berikut.

- Memahami konsep Canny Edge Detection sebagai metode dasar deteksi tepi.
- Menggabungkan metode Canny Edge Detection dengan Instance Segmentation untuk meningkatkan deteksi jalur.
- Menggunakan dataset Rail Segmentation dari Kaggle untuk eksperimen.
- Menggunakan Instance Segmentation untuk deteksi jalur rel kereta (Lane Detection).

3. Landasan Teori

Canny Edge Detection adalah algoritma deteksi tepi yang dirancang untuk mengidentifikasi tepi dalam citra dengan akurasi tinggi. Algoritma ini dikembangkan oleh John F. Canny pada tahun 1986 dan telah menjadi standar dalam pemrosesan citra digital. Canny Edge Detection bekerja melalui beberapa tahapan, termasuk penghalusan citra menggunakan filter Gaussian untuk mengurangi noise, perhitungan gradien intensitas untuk menemukan perubahan intensitas yang signifikan, dan penerapan non-maximum suppression untuk memastikan bahwa hanya tepi lokal maksimum yang dipertahankan. Proses ini diakhiri dengan double thresholding dan edge tracking by hysteresis untuk menghubungkan tepi yang terdeteksi. Algoritma ini terkenal karena kemampuannya dalam mendeteksi tepi yang halus dan akurat, menjadikannya pilihan yang populer dalam berbagai aplikasi pemrosesan citra.

Instance Segmentation adalah teknik yang sangat efektif untuk deteksi jalur rel kereta, karena kemampuannya untuk mengidentifikasi dan memisahkan objek individu dalam citra. Dalam konteks deteksi jalur rel kereta, Instance Segmentation dapat digunakan untuk membedakan antara rel dan elemen lain dalam citra, seperti kerikil atau vegetasi. Dengan melatih model pada dataset yang relevan, seperti Rail Segmentation dari Kaggle, sistem dapat belajar untuk mengenali pola dan karakteristik unik dari jalur rel kereta. Ini memungkinkan deteksi yang lebih akurat dan dapat diandalkan, yang penting untuk aplikasi seperti sistem navigasi kereta otomatis.

Dataset Rail Segmentation dari Kaggle menyediakan data yang kaya dan bervariasi untuk eksperimen deteksi jalur rel kereta. Dataset ini mencakup berbagai citra rel kereta dengan anotasi yang tepat, memungkinkan peneliti untuk melatih dan menguji model deteksi jalur dengan data yang realistis. Penggunaan dataset ini penting untuk memastikan bahwa model yang dikembangkan dapat diandalkan dalam situasi dunia nyata. Dengan data yang teranotasi dengan baik, peneliti dapat mengevaluasi kinerja model secara kuantitatif dan melakukan penyesuaian yang diperlukan untuk meningkatkan akurasi deteksi.

Menggabungkan Canny Edge Detection dengan Instance Segmentation dapat meningkatkan akurasi dan efisiensi dalam deteksi jalur, terutama dalam konteks sistem kendaraan otonom. Instance Segmentation, yang merupakan teknik pembelajaran mendalam, tidak hanya mengklasifikasikan setiap piksel dalam citra tetapi juga membedakan antara objek yang berbeda dari kelas yang sama. Dengan mengintegrasikan Canny Edge Detection, yang efektif dalam mendeteksi tepi, dengan kemampuan Instance Segmentation untuk memisahkan objek, sistem dapat lebih akurat dalam mengidentifikasi dan melacak jalur. Kombinasi ini memungkinkan deteksi jalur yang lebih robust, bahkan dalam kondisi pencahayaan yang buruk atau lingkungan yang kompleks.

GitHub digunakan sebagai platform untuk version control dan dokumentasi proyek. Version control memungkinkan pengembang untuk melacak perubahan dalam kode dan berkolaborasi dengan orang lain secara efektif. Dokumentasi proyek mencakup deskripsi algoritma, implementasi kode, dan hasil eksperimen. Penggunaan GitHub memfasilitasi transparansi dan reproduktibilitas penelitian, serta memungkinkan orang lain untuk membangun di atas pekerjaan yang telah dilakukan. GitHub adalah *toolkit* untuk mengembangkan dan membandingkan algoritma

Python adalah salah satu bahasa pemrograman yang digunakan dalam pengembangan sistem kontrol cerdas. Bahasa pemrograman ini sangat diminati oleh pengguna karena kemudahan penggunaannya dan banyaknya pustaka yang tersedia.

4. Analisis dan Diskusi

Analisis Hasil:

- a. Seberapa baik deteksi jalur dengan Instance Segmentation?

Jawab:

Deteksi jalur dengan Instance Segmentation umumnya sangat baik, terutama dalam kondisi di mana jalur harus dibedakan dari latar belakang yang kompleks. Instance Segmentation mampu mengidentifikasi dan memisahkan setiap objek individu dalam citra, termasuk jalur, dengan presisi tinggi. Ini memungkinkan deteksi yang lebih akurat dibandingkan metode deteksi jalur tradisional, terutama dalam situasi di mana jalur tumpang tindih atau terdapat banyak gangguan visual.

- b. Apakah kombinasi kedua metode dapat meningkatkan akurasi?

Jawab:

Ya, kombinasi Canny Edge Detection dengan Instance Segmentation dapat meningkatkan akurasi deteksi jalur. Canny Edge Detection efektif dalam mendeteksi tepi yang halus dan akurat, sementara Instance Segmentation dapat memisahkan dan mengidentifikasi objek individu. Dengan menggabungkan kedua metode ini, sistem dapat memanfaatkan kekuatan masing-masing untuk meningkatkan deteksi jalur, terutama dalam kondisi pencahayaan yang buruk atau lingkungan yang kompleks.

- c. Apa dampak perubahan parameter Canny (thresholds) terhadap hasil deteksi?

Jawab:

Perubahan parameter threshold pada Canny Edge Detection dapat secara signifikan mempengaruhi hasil deteksi. Threshold yang lebih rendah dapat menyebabkan lebih banyak tepi terdeteksi, termasuk noise, sementara threshold yang lebih tinggi dapat mengabaikan tepi yang lebih halus. Oleh karena itu, pemilihan threshold yang tepat sangat penting untuk mencapai keseimbangan antara mendeteksi tepi yang relevan dan meminimalkan deteksi tepi palsu.

Diskusi:

- a. Kapan lebih baik menggunakan Canny Edge Detection dibanding Instance Segmentation?

Jawab:

Canny Edge Detection lebih baik digunakan dalam situasi di mana deteksi tepi yang cepat dan efisien diperlukan, dan ketika objek dalam citra memiliki kontras yang jelas dengan latar belakang. Ini juga lebih cocok untuk aplikasi dengan sumber daya komputasi terbatas, karena Canny Edge Detection umumnya lebih ringan dibandingkan Instance Segmentation.

- b. Bagaimana cara meningkatkan deteksi jalur dengan tuning parameter YOLOv8-seg?

Jawab:

Untuk meningkatkan deteksi jalur dengan YOLOv8-seg, tuning parameter seperti learning rate, batch size, dan anchor boxes dapat dilakukan. Selain itu, augmentasi data dan penggunaan dataset yang lebih beragam dapat membantu model belajar lebih baik. Memperhatikan arsitektur model dan menyesuaikannya dengan kebutuhan spesifik juga dapat meningkatkan kinerja deteksi.

- c. Bagaimana metode ini dapat diterapkan dalam sistem navigasi kereta otomatis?

Jawab:

Metode ini dapat diterapkan dalam sistem navigasi kereta otomatis dengan menyediakan deteksi jalur yang akurat dan real-time, yang penting untuk navigasi yang aman dan efisien. Dengan mengintegrasikan deteksi jalur berbasis Instance Segmentation dan Canny Edge Detection, sistem dapat lebih baik dalam mengenali jalur rel dan menghindari potensi bahaya. Ini juga memungkinkan sistem untuk beradaptasi dengan berbagai kondisi lingkungan, seperti perubahan cuaca atau rintangan di jalur.

5. Assignment

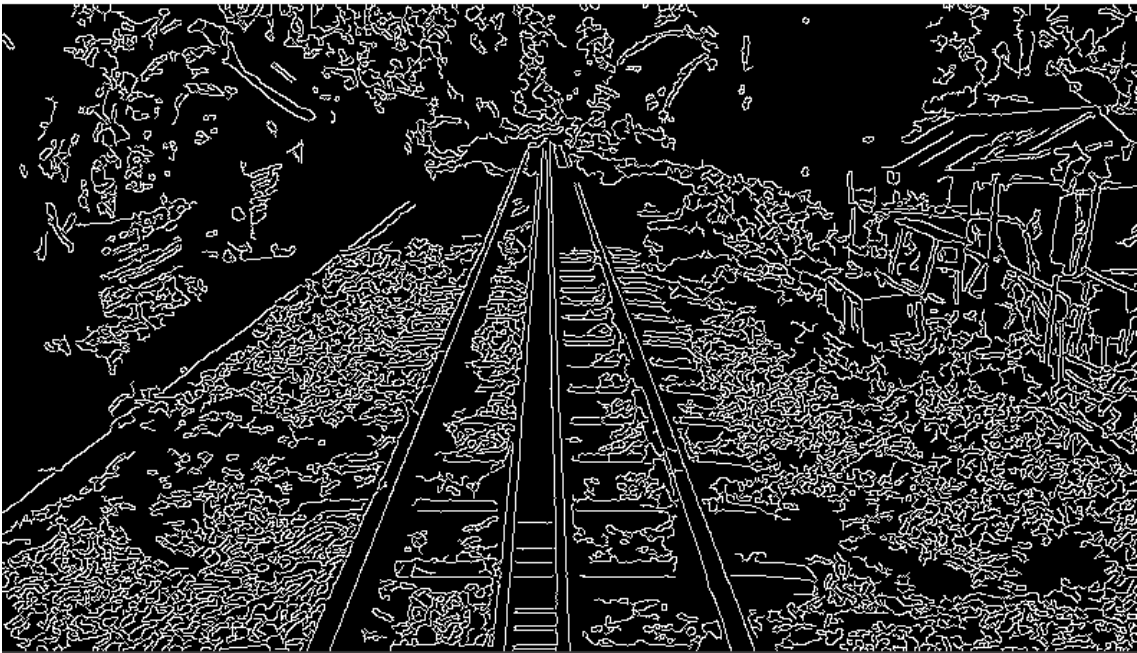
Topik percobaan ini adalah mengubah parameter Canny Edge Detection dan membandingkan hasilnya. Kemudian, memodifikasi model YOLOv8-seg agar hanya mendeteksi jalur rel.

Hasil dari percobaan praktikum ini yaitu program dapat bekerja dengan baik. Kode program ini bertujuan untuk mempersiapkan sistem deteksi objek menggunakan model YOLO

dari Ultralytics, yang dirancang untuk mendeteksi objek rel. Program dimulai dengan mengimpor pustaka yang diperlukan, termasuk OpenCV untuk pemrosesan citra, NumPy untuk manipulasi array, dan PyTorch untuk mendukung pembelajaran mendalam. Selanjutnya, program menentukan lokasi file model terlatih YOLO yang disimpan dalam format '.pt'. Dengan menggunakan perintah 'YOLO(MODEL_PATH)', program mencoba memuat model dari path yang telah ditentukan. Jika model berhasil dimuat, program akan mencetak pesan "Model berhasil dimuat!" ke terminal. Namun, jika terjadi kesalahan saat memuat model, seperti file tidak ditemukan atau kesalahan lainnya, program akan menangkap exception dan mencetak pesan kesalahan yang relevan.

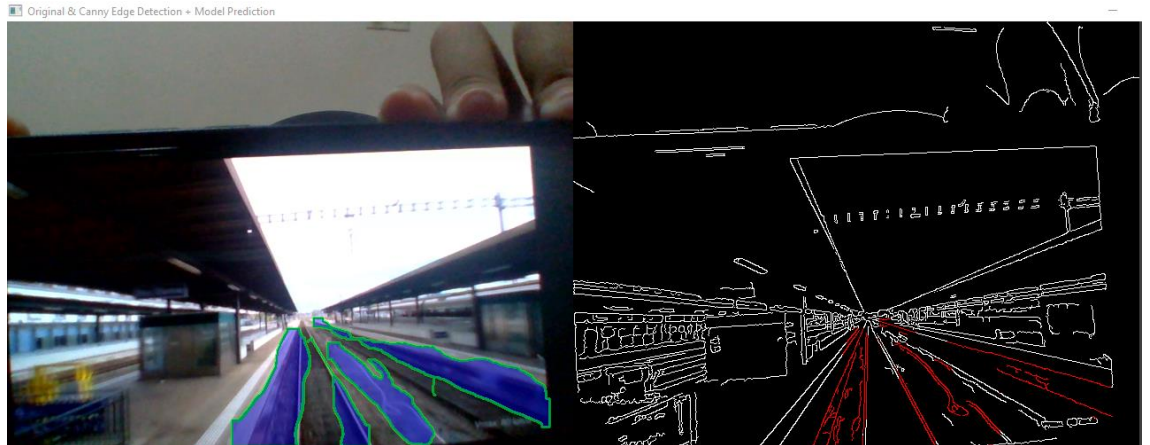
6. Data dan Output Hasil Pengamatan

Sajikan data dan hasil yang diperoleh selama percobaan. Gunakan tabel untuk menyajikan data jika diperlukan.

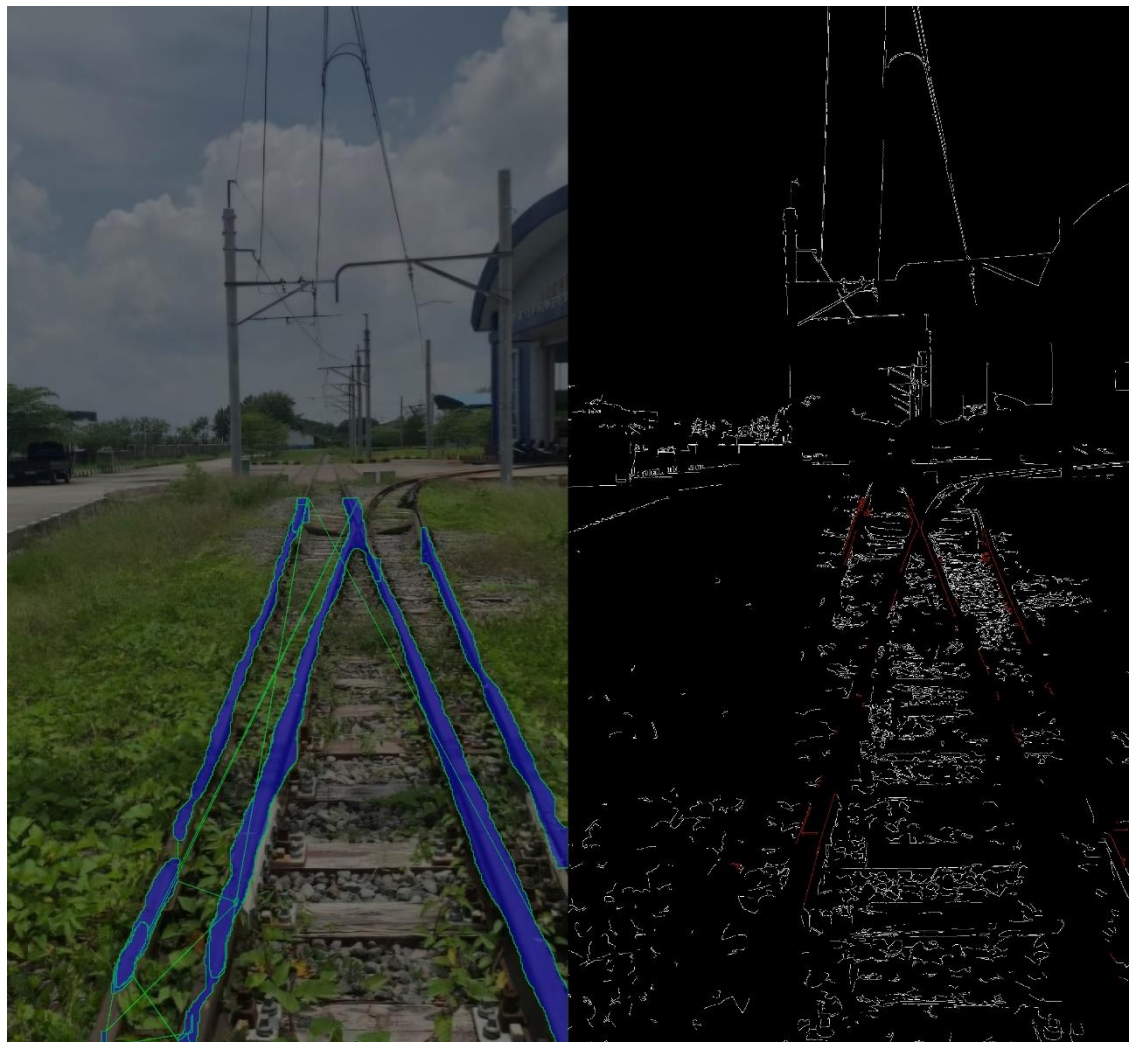
No	Variabel	Hasil Pengamatan
1	Canny Edge Detection	Output yang ditampilkan dapat berjalan dengan baik. 
2	Lane Detection	Output yang ditampilkan di bawah ini.



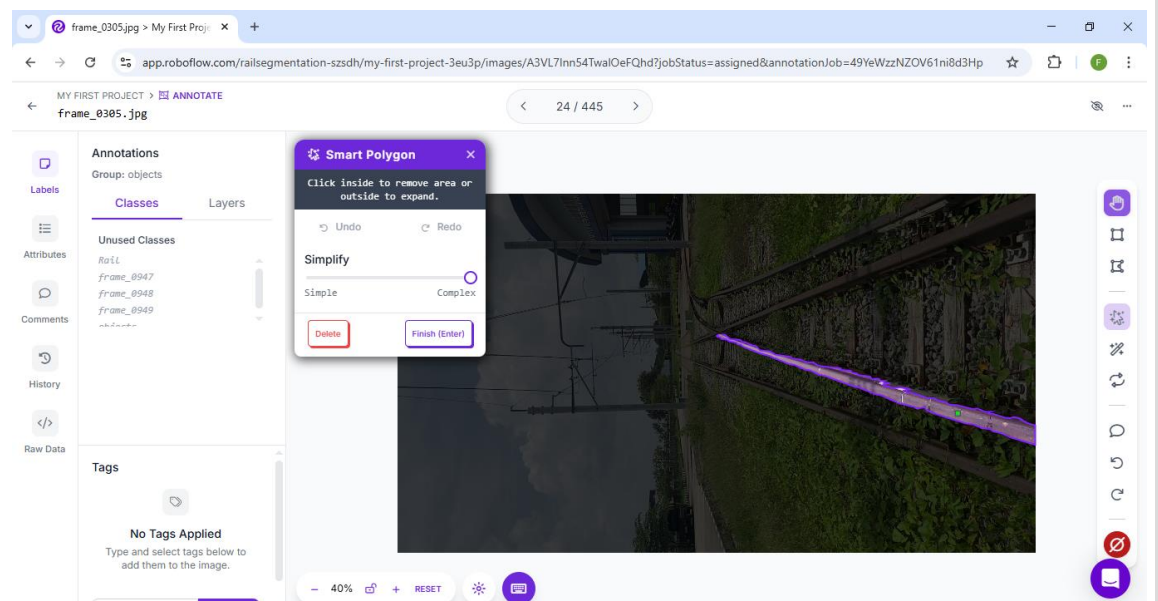
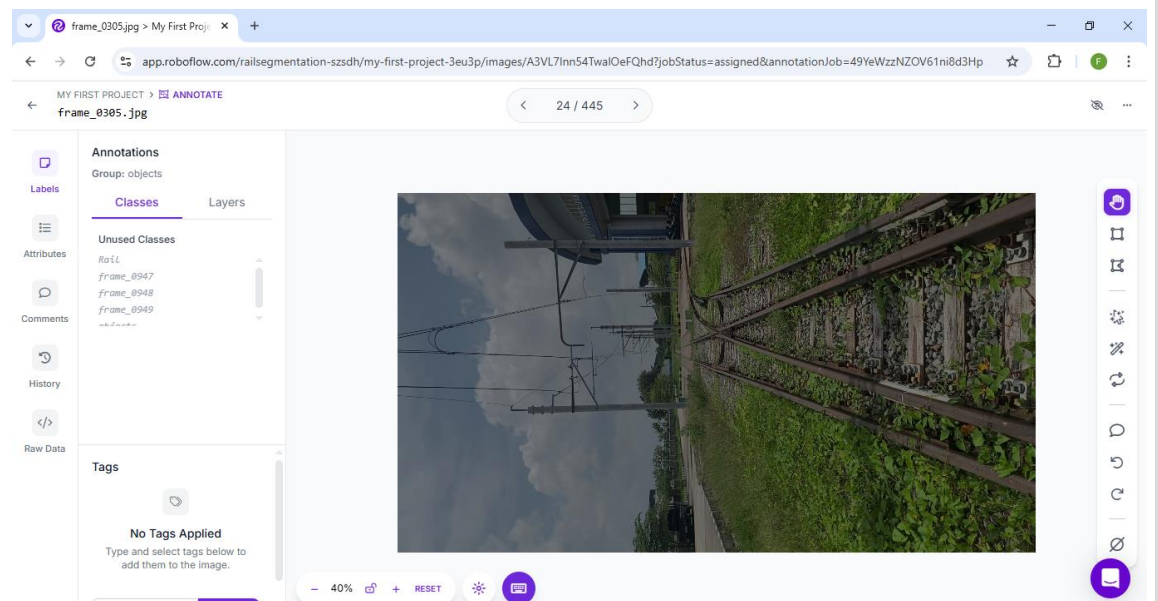
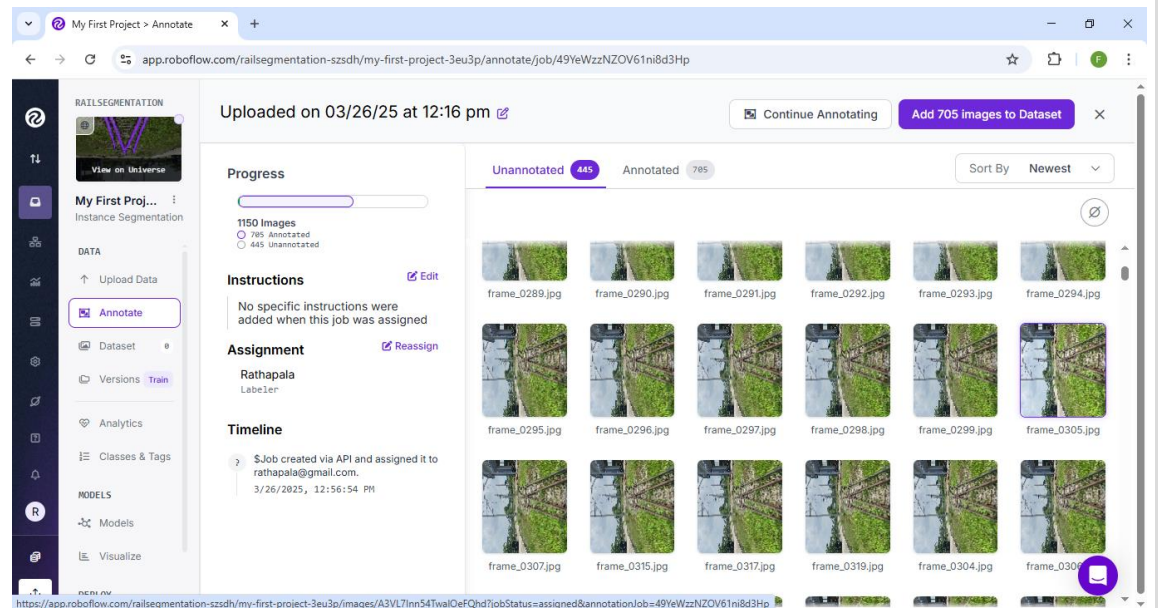
3 Combined Detection
Output yang ditampilkan dapat berjalan dengan baik.

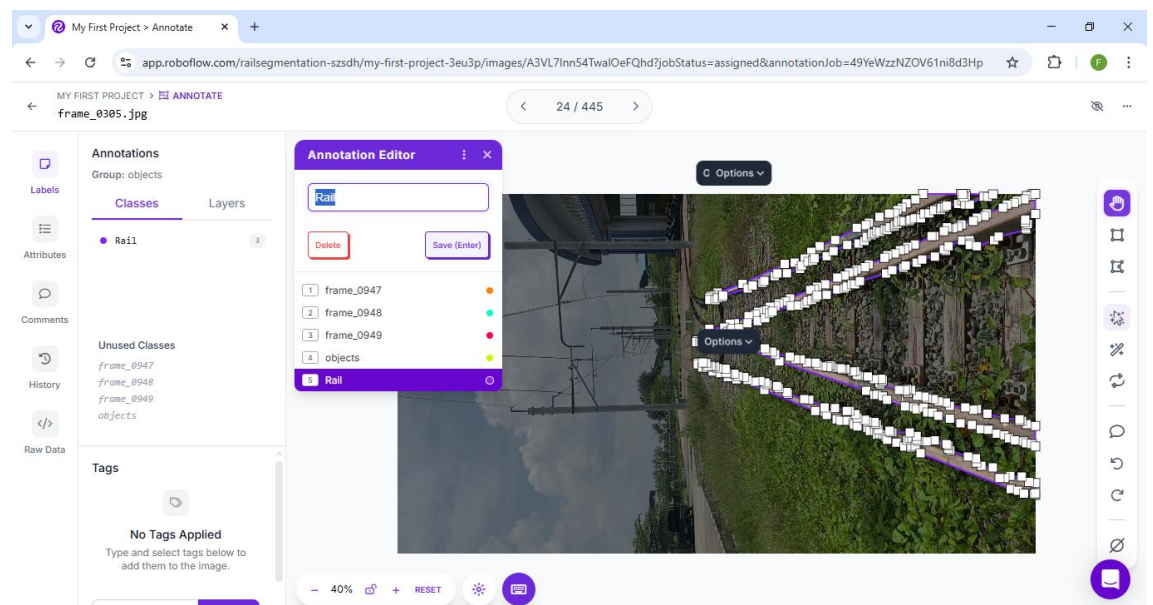
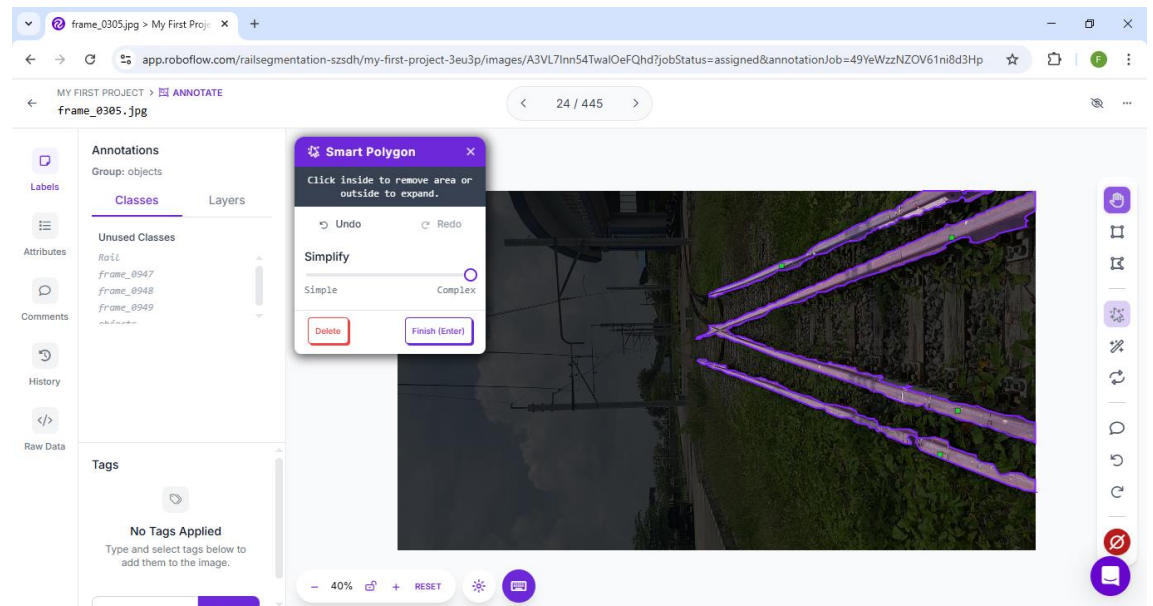


4 Combined Detection
Dengan Dataset Berbeda (Dataset Rail)
Output yang ditampilkan berjalan dengan baik.



Proses dataset rail di bawah ini.





7. Kesimpulan

Berdasarkan percobaan dan analisis data dapat disimpulkan bahwa

- Canny Edge Detection adalah algoritma deteksi tepi yang dirancang untuk mengidentifikasi tepi dalam citra dengan akurasi tinggi.
- Instance Segmentation adalah teknik yang sangat efektif untuk deteksi jalur rel kereta, karena kemampuannya untuk mengidentifikasi dan memisahkan objek individu dalam citra.
- Kode program Canny Edge Detection, Lane Detection, dan Combined dapat dijalankan dengan baik.
- Kombinasi Canny Edge Detection dengan Instance Segmentation dapat meningkatkan akurasi deteksi jalur.

8. Saran

Menambahkan kode program Lane Detection agar dapat digunakan tanpa perangkat NVIDIA.

9. Daftar Pustaka

- Muttaqin, Arafah M., Jaya A.K., Suryawan M.A., Gustiana Z., Banjarnahor A.R., Bukidz D.P., Hazriani, Simanjuntak M., Saputra N., Fajrillah. 2023. *Implementasi Artificial Intelligence (AI) dalam Kehidupan*. Cetakan ke-1. Yayasan Kita Menulis. Langsa.
- Santoso J.T., 2023. *KECERDASAN BUATAN (Artificial Intelligence)*. Yayasan Prima Agus Teknik Bekerja sama dengan Universitas Sains & Teknologi Komputer (Universitas STEKOM). Semarang.
- Zhang, Y., Lu, Z., Zhang, X., Xue, J. H., & Liao, Q. (2021). Deep learning in lane marking detection: A survey. *IEEE Transactions on Intelligent Transportation Systems*, 23(7), 5976-5992.

LAMPIRAN

1. Program Canny Edge Detection

```
import cv2

import numpy as np

def canny_edge_detection(image_path):

    """Mendeteksi tepi menggunakan metode Canny Edge Detection"""

    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    img_blur = cv2.GaussianBlur(img, (5, 5), 0)

    edges = cv2.Canny(img_blur, 50, 150)

    cv2.imshow("Canny Edge Detection", edges)

    cv2.waitKey(0)

    cv2.destroyAllWindows()

    cv2.imwrite("canny_result.jpg", edges)

    return "canny_result.jpg"

# Contoh penggunaan

canny_edge_detection("dataset.jpg")
```

2. Program Lane Detection

```
from ultralytics import YOLO

import cv2

# Load model YOLOv8 Instance Segmentation

model = YOLO("yolov8n-seg.pt")

def detect_rail_lane(image_path):

    """Mendeteksi jalur rel menggunakan YOLOv8 Instance Segmentation"""

    results = model(image_path, show=True)

    results[0].save("lane_detection_result.jpg")

# Contoh penggunaan

detect_rail_lane("dataset.jpg")

from ultralytics import YOLO

import cv2

# Load model YOLOv8 Instance Segmentation

model = YOLO("yolov8n-seg.pt")

def detect_rail_lane(image_path):

    """Mendeteksi jalur rel menggunakan YOLOv8 Instance Segmentation"""

    results = model(image_path, show=True)

    results[0].save("lane_detection_result.jpg")
```

```
# Contoh penggunaan
```

```
detect_rail_lane("dataset.jpg")
```

3. Program Combined Detection

```
from ultralytics import YOLO
```

```
import cv2
```

```
import numpy as np
```

```
from canny_edge import canny_edge_detection
```

```
# Load YOLOv8 Instance Segmentation model
```

```
model = YOLO("yolov8n-seg.pt")
```

```
def combined_detection(image_path):
```

```
    """Menggabungkan Canny Edge Detection dengan Lane Detection"""
```

```
    # Jalankan Canny Edge Detection
```

```
    canny_result = canny_edge_detection(image_path)
```

```
    # Jalankan Lane Detection dengan YOLOv8-seg
```

```
    results = model(image_path)
```

```
    lane_img = results[0].plot()
```

```
    # Baca hasil Canny Edge Detection
```

```
    edges = cv2.imread(canny_result, cv2.IMREAD_GRAYSCALE)
```

Overlay hasil Lane Detection dengan Canny Edge

```
combined = cv2.addWeighted(lane_img, 0.7, cv2.cvtColor(edges, cv2.COLOR_GRAY2BGR),  
0.3, 0)
```

Simpan dan tampilkan hasil

```
cv2.imshow("Combined Detection", combined)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

```
cv2.imwrite("combined_result.jpg", combined)
```

Contoh penggunaan

```
combined_detection("dataset.jpg")
```